

NAVFarm — Piggery Programme Status

As at 2 September 2026. Every figure below was measured against the running system, not estimated. Where something is a projection rather than a fact, it says so.

1. What NAVFarm is, in business terms

NAVFarm is a **multi-tenant livestock ERP**. It is not a herd tracker with reports bolted on — the accounting is the point. A pig farm's real question is *what did this batch cost me, and where did the money go*, and answering that means the animal movements and the general ledger have to be the same system.

Three ideas carry the design:

Nature of Business → Line of Business. A NOB is the broad vertical (LIVESTOCK, POULTRY, AGRI, AQUA, INSECT, PRODUCTION); a LOB is the specific operation inside it. Piggery and Dairy sit under Livestock; Hatchery, Broiler and Layer under Poultry. The database ships **16 LOBs across 6 NOBs** as taxonomy.

One tenant, many companies, many operational areas. A tenant is the customer. Inside it, each legal entity is a company with its own ledger; inside a company, an operational area is a physical unit running one LOB. The demo tenant has two companies — a nucleus breeding business and a commercial grow-finish business — with one piggery area each.

Batches carry cost; animals carry identity. A batch accumulates feed, medicine, labour and overhead, and closes into finished inventory. An animal register row carries the ear tag, the parity count and its own stage and pen. They are joined, not merged — which is what makes "move most of the cohort but hold two sows back" expressible at all.

Why piggery first

Only LVS_PIGGERY is in scope. The other fifteen LOBs exist as taxonomy rows and nothing else: **piggery has 11 lifecycle stages configured; every other LOB has zero**. Since batches, data entry, records, schedulers and costing all hang off stage_master, those LOBs are non-functional *by decision*, not by oversight. They get built once piggery is complete and its operational area does everything a real pig farm needs.

The core — batch, stage, parameter, scheduler, costing, inventory, GL — is already LOB-generic. That is what makes adding a line of business later cheap: a set of stages and a seed, not a new subsystem.

2. What the system covers today

Scale

Tenant database tables	102
Migrations applied	51

API operations	387
Enforced permission pairs	44
Console languages	8
Lifecycle stages (piggery)	11

The three workspace scopes

The console renders as three different products depending on where the user stands. Scope is derived from user type, not chosen freely, and the API re-checks every request regardless.

- **Tenant** — consolidation across companies: census, WIP, the companies register, users, roles, audit trail, notification channels.
- **Company** — the legal entity: journal, trial balance, P&L, balance sheet, IAS 41 bio-asset statement, batch cost variance, inventory balances per warehouse.
- **Operational** — the day's work: batches, stages, data entry, records, transfers, livestock, alerts, approvals, traceability, area settings.

Piggery lifecycle

Eleven stages from Quarantine through Gilt Grower, Flush/Service, Gestation, Farrowing, Lactation, Weaning, Boar AI, CB Grower, Slaughter to Disposed. Sequence, durations, minimum days before a move and transition triggers are **data, not code**. A minimum-days rule stops a batch being advanced out of gestation on day three, and an alternate-next-stage rule makes the backward move legal: gestation's normal next stage is Farrowing, but under `PREGNANCY_FAILED` it is Flush/Service.

Partial stage moves — the case that shaped the design

A cohort of twenty sows where two fail the day-35 scan. The rest must go forward; those two must go back. Verified working end to end:

1. Split the two out — a **child batch with an explicit parent link**, held at the earlier stage, in a different pen.
2. The parent advances normally; its cascade only touches animals still in the stage being left, so the held-back pair is untouched.
3. Merge them back when ready — every live animal returns to the parent **in step with the parent's current stage**, and the child closes.

They can also be transferred onward to a different batch instead of merged. Both batches share one scheduler, and because schedule lines are filtered by the batch's current stage, the parent gets gestation lines while the child gets flush lines — one scheduler, two stages, two plans.

Costing

Two methods, deliberately different:

- **BIO_ASSET** — the breeding herd, IAS 41 biological assets. Carrying value, amortisation, fair-value adjustment, harvest and disposal all post to a bio-asset ledger. Such a batch cannot be "closed"; it ends when its last animal leaves the herd.
- **STANDARD** — grow-finish batches. Cost accumulates, then closing values the output at standard and posts the difference as price, usage, output and overhead variances. **The close refuses if it does not reconcile to within ₹0.01** — nothing is written unless actual cost equals standard output value plus variances.

Also live: inventory with FIFO layers, goods receipt/issue/transfer/adjustment, general ledger with configurable mappings, QC parameters and inspections, approvals, alerts driven by scheduler KPI thresholds, and pack-level QR traceability carrying an origin batch chain.

3. What was done in this engagement

Verified in the running application, not inferred from test output.

Data integrity and accounting

AREA	PROBLEM FOUND	RESOLUTION
Standard costing	<code>batch_standard</code> seeded with five columns that do not exist — Drizzle dropped them all, so every standard was <code>NULL</code>	Baselines derived from actuals so a close reconciles exactly
Closed batch	<code>CLOSED</code> written straight to the row, skipping <code>close()</code> — no output valuation, no cost, no variances	Batch staged active; closing it in the app posts everything
Variance report	Summed <code>std_value</code> / <code>actual_value</code> across types where they hold a rate, a quantity and an amount	Cost columns derived from the batch's own accumulated cost
IAS 41 statement	Seed wrote <code>APPRECIATION</code> / <code>AMORTISATION</code> , which the roll-forward does not classify — fair value was reported as acquisitions, amortisation as disposals	Corrected entry types; guard test so no writer can emit an unclassified type
Bio-asset harvest	<code>OUTPUT</code> on a matured bio-asset batch reduced carrying value with no ledger row — the harvest bucket was structurally zero	Writes a <code>TRANSFORMATION</code> movement; verified the bucket moves
Approvals list	DTO advertised <code>limit</code> / <code>offset</code> ; the service ignored both and returned every row	Paginates; badges moved to the counts endpoint so they stay correct
WIP tile	Structurally zero for every open batch	Computed from consumption and overhead
Animal lookup	Returned <code>batch_code</code> , which is not a column — always <code>undefined</code>	Returns <code>batch_no</code>

Clinical records

Mortality and treatment detail — cause of death, post-mortem findings, disposal method, diagnosis, route, withdrawal period, vet — was formatted into a free-text `remarks` string and parsed back by the console. It is now two proper tables keyed 1:1 to the transaction, validated **before** anything is written, with `withdrawal_days` driving the active-case count and the slaughter withdrawal check.

Correctness of the console

Static content that pretended to be data: the batch stages page rendered a hardcoded array with status assigned by row position; barn climate was a literal string; the health page invented mortality rate, vaccination coverage, biosecurity grade, and per-row pen, disposal, route, vet and withdrawal. All now computed or read from what was recorded. A stage transition could write a display code that matched no stage row — the API now rejects it.

Access control and navigation

- **Six permissions the API enforced could not be granted by any role** — including `PIGGERY/ANIMAL`, the animal register. Added, with a guard test comparing the roles screen against every `@RequirePermission` in the API.
- **Sidebar labels disagreed across scopes** — the same route was "Finance" in one scope and "Finance & Costing" in another, "Production" versus "Batches". Unified, with company scope gaining the same expandable groups, child lists extracted so they cannot drift, and a guard test.
- A dead session left the user on a dashboard of zeroes; it now signs out and redirects.

Scope hygiene

- Non-piggery LOBs no longer seed items, breeds or parameters — **0 rows each**, while piggery keeps its 33/4/35. Done by filtering, so enabling a LOB later is one array entry rather than re-typing definitions.
- `nob_lob_extension_config` dropped — zero rows, zero code references.
- Two redundant operational-area endpoints deleted (387 operations, down from 389).
- **API typecheck went from 48 errors to 0**, including twenty dead `success: true` literals overwritten by a spread and a KPI block reading a relation that is never loaded.

Current gates

GATE	RESULT
API tests	222 / 222
Web tests	80 / 80
API typecheck	0 errors
Web typecheck	0 errors
API lint	0 errors
Web lint	87 errors (accepted baseline, none new)

4. What is left

A. Designed and approved, not yet built

First-class farm records — [full design](#). Today an entry against twelve selected animals writes twelve unrelated rows with no handle to edit as one thing, and **nothing in the system is editable at all**. The design introduces a `farm_record` that a person creates and corrects, with transactions as its accounting consequence; same-day edits for supervisors, unrestricted for admins on an open batch and narrative-only once it closes; and reverse-and-repost so posted history is never rewritten.

A demo farm that looks like a real one. The current seed has 39 animals with **zero sires, zero dams and zero dates of birth** — sows carry parity numbers with no litters behind them. The replacement is four generations over roughly two years where the numbers reconcile: parity equals litter count, every born animal traces to a farrowing, every farrowing to a mating ~114 days earlier.

B. Small gaps identified, not yet closed

- **Resource maintenance has no UI at all.** Five endpoints, no screen — logs exist in the data and are invisible. Larger than first reported.
- `GET /role/assignments/{userId}` **is redundant** — the users page already receives each user's role. Should be deleted.
- `GET /scheduler/suggest-lines` **is unused**. It proposes schedule lines from breed lifecycle standards; today every line is built by hand.

C. Deliberately deferred

- **Reopening a closed batch** — needed to correct a *number* after close. Requires reversing output valuation, inventory receipt, variance rows and journals, with guards for output already sold or packed.
- **Records for animals outside a batch** — a death in the breeding herd currently has nowhere to go, because a record requires a batch.

D. Not started, by decision

- The other fifteen lines of business.
- The mobile app. `apps/mobile` is a bare Flutter scaffold — a single `main.dart`.

5. Suggestions

Finish the operational area before widening. The most valuable remaining work is the records layer: it is what turns the console from a system that records into one that can be corrected, and every LOB added later inherits it. Widening to poultry first would multiply an unfinished foundation.

Treat the seed as a test fixture, not decoration. Most defects in this engagement were found because seeded data disagreed with itself — standards that were null, entry types nothing classified, parity without litters. A seed whose numbers reconcile is a continuous check on the application.

Keep adding cross-boundary guard tests. Four bugs here were two components disagreeing: roles UI versus enforced permissions, sidebar versus routes, seed versus report, nav versus pages. Each is now a test that reads

both sides and fails on drift. That pattern is cheap and catches a class of bug no unit test would.

When a LOB is next enabled, start with `stage_master`. Stages are the spine — batches, data entry, records, schedulers and costing all hang off them. Seeding a lifecycle turns a dead LOB into a working one on the existing engine, with no new code.

Two operational notes. The 8 GB development machine needs the workspace excludes now in `.vscode/settings.json`, and dev servers stopped when not in use. And the web lint baseline of 87 errors should be driven down deliberately at some point rather than carried indefinitely.

6. Timeline

Estimates are in developer-days for one developer familiar with this codebase, and assume the current machine and no parallel workstreams. They are projections, not commitments.

#	WORK	ESTIMATE	DEPENDS ON
1	Close remaining small gaps — maintenance UI, delete the redundant endpoint, wire <code>suggest-lines</code>	3–4 d	—
2	First-class records: schema, migration, service, reverse-and-repost, backfill	6–8 d	—
3	Data entry and records console rebuilt on records, with editing	4–5 d	2
4	Multi-generation demo farm	3–4 d	2
5	End-to-end verification, documentation refresh	2–3 d	2, 3, 4
	Piggery feature-complete	18–24 d	
6	Reopen-batch flow	4–5 d	2
7	Records for animals outside a batch	2–3 d	2
	Piggery complete including deferred items	24–32 d	

At five working days a week that is roughly **four to five weeks** to feature-complete, or **five to seven weeks** including the deferred items.

What would move these numbers. The records layer (items 2 and 3) is the only piece with real unknowns — it touches costing, the inventory ledger, the GL and KPI alerts, and three traps are already documented in the design (FIFO reversal that must unwind applications rather than post a contra entry, KPI alerts that must be withdrawn, and bio-asset state that must be recomputed rather than inverted). Those are understood, which is why the estimate is a range rather than a guess, but they are where slippage would come from.

Adding a second line of business is a separate programme and is not estimated here. On the current architecture the marginal cost of a LOB is its stages, its master data and any species-specific screens — materially less than piggery cost, but not small.

7. Running it

```
pnpm nx run api:db-seed-demo --args="--fresh" # rebuilds the three databases
pnpm nx serve api # :2877
pnpm nx serve web # :3002
```

`--fresh` drops only the three NAVFarm databases by name. The seed is idempotent — run without `--fresh` it repairs and backfills in place.

Sign in with any account below; the password is `12345678`.

USER	TYPE	SEES
<code>admin@apexagri.local</code>	Tenant admin	Both companies, all scopes
<code>arjun.sharma@apexagri.local</code>	Company admin	Apex
<code>vikram.singh@highlandpork.local</code>	Company admin	Highland
<code>supervisor@apexpork.local</code>	Operational admin	Apex piggery area
<code>supervisor@highlandpork.local</code>	Operational admin	Highland piggery area

To see the full standard-costing flow, sign in as the Highland company admin and close `PIG-BAT-2026-0102` at 98 head with an actual end date of 2026-07-15. That posts the output valuation, six variance rows and their journals, and fills the Batch Cost Variance report.

Related documents

- [Operations guide](#) — how the system works, with screenshots of all three scopes
- [First-class records design](#) — the approved design for the next phase